



# Reinforcement Learning

---

P Satya Jayadev

Web Enabled M.Tech,  
Indian Institute of Technology Madras

# Topic 6

## Actor-Critic Methods



1. From REINFORCE to Actor–Critic
2. Actor and Critic Components
3. TD Error and Advantage Estimation
4. One-Step Actor–Critic Algorithm
5. Parallel Actor–Critic: A2C and A3C

**References:** Sutton, R. S., Barto, A. G. (2018 ). Reinforcement Learning: An Introduction. The MIT Press.



# Why Move Beyond REINFORCE?

REINFORCE updates the policy using the sampled return:

$$\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

Because  $G_t$  depends on all subsequent rewards, it can vary considerably between trajectories. A value baseline produces a better update:

$$\theta \leftarrow \theta + \alpha \gamma^t (G_t - \hat{V}(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$



# Why Move Beyond REINFORCE?

REINFORCE updates the policy using the sampled return:

$$\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

Because  $G_t$  depends on all subsequent rewards, it can vary considerably between trajectories. A value baseline produces a better update:

$$\theta \leftarrow \theta + \alpha \gamma^t (G_t - \hat{V}(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

## What the Baseline Improves

- ▶ Compares the observed return with the expected return.
- ▶ Reduces variance relative to using  $G_t$  alone.

## What Still Remains

- ▶  $G_t$  is still a sampled Monte Carlo return.
- ▶ It is available only after those rewards are observed.



# Why Move Beyond REINFORCE?

REINFORCE updates the policy using the sampled return:

$$\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

Because  $G_t$  depends on all subsequent rewards, it can vary considerably between trajectories. A value baseline produces a better update:

$$\theta \leftarrow \theta + \alpha \gamma^t (G_t - \hat{V}(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

## What the Baseline Improves

- ▶ Compares the observed return with the expected return.
- ▶ Reduces variance relative to using  $G_t$  alone.

## What Still Remains

- ▶  $G_t$  is still a sampled Monte Carlo return.
- ▶ It is available only after those rewards are observed.

## Motivation for Actor–Critic

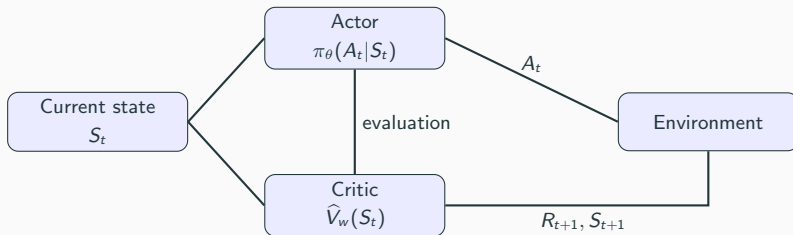
Learn the value function as a critic and use one-step TD feedback instead of relying entirely on the complete sampled return  $G_t$ .



# Actor-Critic Methods

---

# Actor–Critic Architecture



## Actor

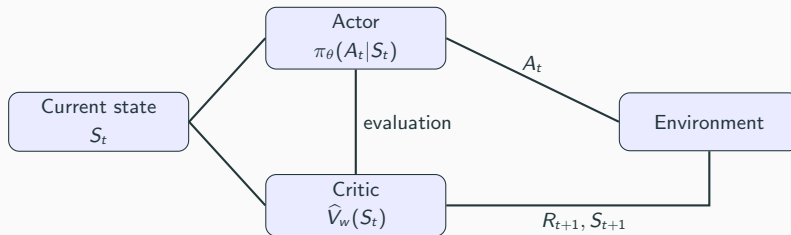
- Represents the policy and selects actions.
- Updates the policy parameters  $\theta$  using critic feedback.

## Critic

- Estimates expected future return.
- Evaluates actions and updates value parameters *wbf*.



# Actor–Critic Architecture



## Actor

- Represents the policy and selects actions.
- Updates the policy parameters  $\theta$  using critic feedback.

## Critic

- Estimates expected future return.
- Evaluates actions and updates value parameters *wbf*.

## Important

The actor and critic are two learned components of the same agent, not two separate agents.



# From the Value Baseline to Advantage

In the policy-gradient topic, we established that the sampled return estimates the action value:

$$q^{\pi}(S_t, A_t) \approx G_t$$

We also introduced the state value as a baseline to reduce variance in target:

$$G_t - \hat{V}(S_t)$$



# From the Value Baseline to Advantage

In the policy-gradient topic, we established that the sampled return estimates the action value:

$$q^{\pi}(S_t, A_t) \approx G_t$$

We also introduced the state value as a baseline to reduce variance in target:

$$G_t - \hat{V}(S_t)$$

Combining these two ideas:

$$G_t - \hat{V}(S_t) \approx q^{\pi}(S_t, A_t) - V^{\pi}(S_t)$$

This difference is called the **advantage**:

$$A^{\pi}(s, a) = q^{\pi}(s, a) - V^{\pi}(s)$$



# From the Value Baseline to Advantage

In the policy-gradient topic, we established that the sampled return estimates the action value:

$$q^\pi(S_t, A_t) \approx G_t$$

We also introduced the state value as a baseline to reduce variance in target:

$$G_t - \hat{V}(S_t)$$

Combining these two ideas:

$$G_t - \hat{V}(S_t) \approx q^\pi(S_t, A_t) - V^\pi(S_t)$$

This difference is called the **advantage**:

$$A^\pi(s, a) = q^\pi(s, a) - V^\pi(s)$$

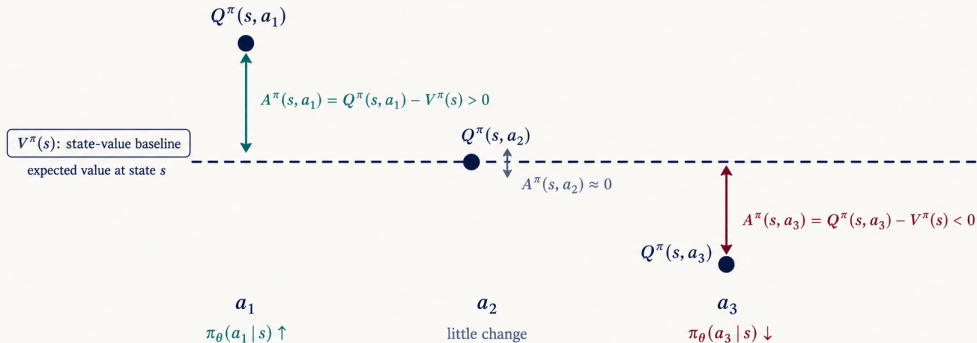
- ▶  $A^\pi(s, a) > 0$ : action  $a$  is better than the policy normally performs in state  $s$ .
- ▶  $A^\pi(s, a) < 0$ : action  $a$  is worse than the policy normally performs in state  $s$ .
- ▶  $A^\pi(s, a) \approx 0$ : action  $a$  performs close to expectation.



# Visualisation: Advantage Function

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

- State value  $V^\pi(s)$
- Action value  $Q^\pi(s, a)$
- ↕ Advantage  $A^\pi(s, a)$



# From Monte Carlo Advantage to TD Advantage

REINFORCE with a baseline estimates advantage using:

$$\hat{A}_t^{\text{MC}} = G_t - \hat{V}_w(S_t)$$



# From Monte Carlo Advantage to TD Advantage

REINFORCE with a baseline estimates advantage using:

$$\hat{A}_t^{\text{MC}} = G_t - \hat{V}_w(S_t)$$

This estimate depends on all rewards received after time  $t$ . Actor-critic methods instead construct a one-step TD target:

$$\text{TD target} = R_{t+1} + \gamma \hat{V}_w(S_{t+1})$$



# From Monte Carlo Advantage to TD Advantage

REINFORCE with a baseline estimates advantage using:

$$\hat{A}_t^{\text{MC}} = G_t - \hat{V}_w(S_t)$$

This estimate depends on all rewards received after time  $t$ . Actor-critic methods instead construct a one-step TD target:

$$\text{TD target} = R_{t+1} + \gamma \hat{V}_w(S_{t+1})$$

The corresponding TD error is:

$$\delta_t = R_{t+1} + \gamma \hat{V}_w(S_{t+1}) - \hat{V}_w(S_t)$$



# From Monte Carlo Advantage to TD Advantage

REINFORCE with a baseline estimates advantage using:

$$\hat{A}_t^{\text{MC}} = G_t - \hat{V}_w(S_t)$$

This estimate depends on all rewards received after time  $t$ . Actor-critic methods instead construct a one-step TD target:

$$\text{TD target} = R_{t+1} + \gamma \hat{V}_w(S_{t+1})$$

The corresponding TD error is:

$$\delta_t = R_{t+1} + \gamma \hat{V}_w(S_{t+1}) - \hat{V}_w(S_t)$$

Expectation of TD error can be estimated by the TD advantage:

$$\mathbb{E}[\delta_t \mid S_t = s, A_t = a] \approx A^\pi(s, a)$$

Actor-critic methods therefore use the TD advantage in place of MC advantage



## How Is TD Advantage Used in Actor–Critic?

After taking action  $A_t$  in state  $S_t$ , the agent observes  $R_{t+1}$  and  $S_{t+1}$  and computes:

$$\hat{A}_t^{\text{TD}} = \delta_t = R_{t+1} + \gamma \hat{V}_w(S_{t+1}) - \hat{V}_w(S_t)$$



# How Is TD Advantage Used in Actor–Critic?

After taking action  $A_t$  in state  $S_t$ , the agent observes  $R_{t+1}$  and  $S_{t+1}$  and computes:

$$\hat{A}_t^{\text{TD}} = \delta_t = R_{t+1} + \gamma \hat{V}_w(S_{t+1}) - \hat{V}_w(S_t)$$

The same TD error is used to update both components:

## 1. Update the Critic

$$w \leftarrow w + \alpha_c \delta_t \nabla_w \hat{V}_w(S_t)$$

- Moves  $\hat{V}_w(S_t)$  towards the TD target.
- Makes the critic's value estimate more closely match the expected return from state  $S_t$ .

## 2. Update the Actor

$$\theta \leftarrow \theta + \alpha_a \gamma^t \delta_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

- $\delta_t > 0$ : increase the probability of  $A_t$ .
- $\delta_t < 0$ : decrease the probability of  $A_t$ .



# How Is TD Advantage Used in Actor–Critic?

After taking action  $A_t$  in state  $S_t$ , the agent observes  $R_{t+1}$  and  $S_{t+1}$  and computes:

$$\hat{A}_t^{\text{TD}} = \delta_t = R_{t+1} + \gamma \hat{V}_w(S_{t+1}) - \hat{V}_w(S_t)$$

The same TD error is used to update both components:

## 1. Update the Critic

$$w \leftarrow w + \alpha_c \delta_t \nabla_w \hat{V}_w(S_t)$$

- Moves  $\hat{V}_w(S_t)$  towards the TD target.
- Makes the critic's value estimate more closely match the expected return from state  $S_t$ .

## 2. Update the Actor

$$\theta \leftarrow \theta + \alpha_a \gamma^t \delta_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$$

- $\delta_t > 0$ : increase the probability of  $A_t$ .
- $\delta_t < 0$ : decrease the probability of  $A_t$ .

## Actor–Critic Learning Loop

The critic learns to evaluate states more accurately, while the actor uses the critic's TD advantage to improve its action probabilities after every transition.



# One-Step TD Actor–Critic Algorithm

- ▶ **Input:** Differentiable policy  $\pi(a|s, \theta)$ , value function  $\hat{V}(s, \mathbf{w})$ , discount factor  $\gamma$ , and learning rates  $\alpha_a, \alpha_c$ .
- ▶ **Initialise:** Randomly initialise actor parameters  $\theta$  and critic parameters  $\mathbf{w}$ .



# One-Step TD Actor–Critic Algorithm

- ▶ **Input:** Differentiable policy  $\pi(a|s, \theta)$ , value function  $\hat{V}(s, \mathbf{w})$ , discount factor  $\gamma$ , and learning rates  $\alpha_a, \alpha_c$ .
- ▶ **Initialise:** Randomly initialise actor parameters  $\theta$  and critic parameters  $\mathbf{w}$ .
- ▶ Repeat for each episode until convergence:
  1. Observe the current state  $S_t$ .



# One-Step TD Actor–Critic Algorithm

- ▶ **Input:** Differentiable policy  $\pi(a|s, \theta)$ , value function  $\hat{V}(s, \mathbf{w})$ , discount factor  $\gamma$ , and learning rates  $\alpha_a, \alpha_c$ .
- ▶ **Initialise:** Randomly initialise actor parameters  $\theta$  and critic parameters  $\mathbf{w}$ .
- ▶ Repeat for each episode until convergence:
  1. Observe the current state  $S_t$ .
  2. Select an action using the actor:  $A_t \sim \pi(\cdot | S_t, \theta_n)$
  3. Execute  $A_t$  and observe  $R_{t+1}$  and  $S_{t+1}$ .



# One-Step TD Actor–Critic Algorithm

- **Input:** Differentiable policy  $\pi(a|s, \theta)$ , value function  $\hat{V}(s, \mathbf{w})$ , discount factor  $\gamma$ , and learning rates  $\alpha_a, \alpha_c$ .
- **Initialise:** Randomly initialise actor parameters  $\theta$  and critic parameters  $\mathbf{w}$ .
- Repeat for each episode until convergence:
  1. Observe the current state  $S_t$ .
  2. Select an action using the actor:  $A_t \sim \pi(\cdot | S_t, \theta_n)$
  3. Execute  $A_t$  and observe  $R_{t+1}$  and  $S_{t+1}$ .
  4. Compute the TD advantage:  $\delta_t = R_{t+1} + \gamma \hat{V}(S_{t+1}, \mathbf{w}_n) - \hat{V}(S_t, \mathbf{w}_n)$



# One-Step TD Actor–Critic Algorithm

- **Input:** Differentiable policy  $\pi(a|s, \theta)$ , value function  $\hat{V}(s, \mathbf{w})$ , discount factor  $\gamma$ , and learning rates  $\alpha_a, \alpha_c$ .
- **Initialise:** Randomly initialise actor parameters  $\theta$  and critic parameters  $\mathbf{w}$ .
- Repeat for each episode until convergence:
  1. Observe the current state  $S_t$ .
  2. Select an action using the actor:  $A_t \sim \pi(\cdot | S_t, \theta_n)$
  3. Execute  $A_t$  and observe  $R_{t+1}$  and  $S_{t+1}$ .
  4. Compute the TD advantage:  $\delta_t = R_{t+1} + \gamma \hat{V}(S_{t+1}, \mathbf{w}_n) - \hat{V}(S_t, \mathbf{w}_n)$
  5. Update the critic:

$$\mathbf{w}_{n+1} = \mathbf{w}_n + \alpha_c \delta_t \nabla_{\mathbf{w}} \hat{V}(S_t, \mathbf{w}_n)$$



# One-Step TD Actor–Critic Algorithm

- **Input:** Differentiable policy  $\pi(a|s, \theta)$ , value function  $\hat{V}(s, \mathbf{w})$ , discount factor  $\gamma$ , and learning rates  $\alpha_a, \alpha_c$ .
- **Initialise:** Randomly initialise actor parameters  $\theta$  and critic parameters  $\mathbf{w}$ .
- Repeat for each episode until convergence:
  1. Observe the current state  $S_t$ .
  2. Select an action using the actor:  $A_t \sim \pi(\cdot|S_t, \theta_n)$
  3. Execute  $A_t$  and observe  $R_{t+1}$  and  $S_{t+1}$ .
  4. Compute the TD advantage:  $\delta_t = R_{t+1} + \gamma \hat{V}(S_{t+1}, \mathbf{w}_n) - \hat{V}(S_t, \mathbf{w}_n)$
  5. Update the critic:

$$\mathbf{w}_{n+1} = \mathbf{w}_n + \alpha_c \delta_t \nabla_{\mathbf{w}} \hat{V}(S_t, \mathbf{w}_n)$$

6. Update the actor:

$$\theta_{n+1} = \theta_n + \alpha_a \gamma^t \delta_t \nabla_{\theta} \log \pi(A_t|S_t, \theta_n)$$



# One-Step TD Actor–Critic Algorithm

- **Input:** Differentiable policy  $\pi(a|s, \theta)$ , value function  $\hat{V}(s, \mathbf{w})$ , discount factor  $\gamma$ , and learning rates  $\alpha_a, \alpha_c$ .
- **Initialise:** Randomly initialise actor parameters  $\theta$  and critic parameters  $\mathbf{w}$ .
- Repeat for each episode until convergence:
  1. Observe the current state  $S_t$ .
  2. Select an action using the actor:  $A_t \sim \pi(\cdot|S_t, \theta_n)$
  3. Execute  $A_t$  and observe  $R_{t+1}$  and  $S_{t+1}$ .
  4. Compute the TD advantage:  $\delta_t = R_{t+1} + \gamma \hat{V}(S_{t+1}, \mathbf{w}_n) - \hat{V}(S_t, \mathbf{w}_n)$
  5. Update the critic:

$$\mathbf{w}_{n+1} = \mathbf{w}_n + \alpha_c \delta_t \nabla_{\mathbf{w}} \hat{V}(S_t, \mathbf{w}_n)$$

6. Update the actor:

$$\theta_{n+1} = \theta_n + \alpha_a \gamma^t \delta_t \nabla_{\theta} \log \pi(A_t|S_t, \theta_n)$$

## Key Difference from REINFORCE

The actor and critic are updated after every transition using  $\delta_t$ ; the algorithm does not wait to calculate the complete return  $G_t$ .



# Example: Vehicle Cruise Control

## Control Objective

Maintain the desired speed  $v^*$  while avoiding excessive acceleration and braking (optimize fuel use).



# Example: Vehicle Cruise Control

## Control Objective

Maintain the desired speed  $v^*$  while avoiding excessive acceleration and braking (optimize fuel use).



- **State Vector:**  $S_t = [v_t, e_t]$ , where  $e_t = v^* - v_t$
- **Actions:**  $A_t \in [-1, 1]$ , where  $A_t > 0$  applies throttle and  $A_t < 0$  applies braking.



# Example: Vehicle Cruise Control

## Control Objective

Maintain the desired speed  $v^*$  while avoiding excessive acceleration and braking (optimize fuel use).



- **State Vector:**  $S_t = [v_t, e_t]$ , where  $e_t = v^* - v_t$
- **Actions:**  $A_t \in [-1, 1]$ , where  $A_t > 0$  applies throttle and  $A_t < 0$  applies braking.
- **Reward:**  $R_{t+1} = -(v^* - v_{t+1})^2 - \lambda A_t^2$
- The first term penalizes deviation from the desired speed, while the second penalizes large throttle or braking commands. Thus, the agent must balance accurate speed tracking with smooth and energy-efficient control.



# Example: Vehicle Cruise Control

## Control Objective

Maintain the desired speed  $v^*$  while avoiding excessive acceleration and braking (optimize fuel use).



- ▶ **State Vector:**  $S_t = [v_t, e_t]$ , where  $e_t = v^* - v_t$
- ▶ **Actions:**  $A_t \in [-1, 1]$ , where  $A_t > 0$  applies throttle and  $A_t < 0$  applies braking.
- ▶ **Reward:**  $R_{t+1} = -(v^* - v_{t+1})^2 - \lambda A_t^2$
- ▶ The first term penalizes deviation from the desired speed, while the second penalizes large throttle or braking commands. Thus, the agent must balance accurate speed tracking with smooth and energy-efficient control.

## What Should the Agent Learn?

Accelerate when the vehicle is too slow, reduce throttle near the target speed and brake when it is too fast.



# Cruise Control Using Actor–Critic



**Critic: Was the Action Better Than Expected?**

$$\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t).$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha_w \delta_t \nabla_{\mathbf{w}} V_w(S_t).$$

**Actor: Modify the Control Policy**

$$\theta \leftarrow \theta + \alpha_\theta \delta_t \nabla_\theta \log \pi_\theta(A_t|S_t).$$



# Cruise Control Using Actor–Critic



**Critic: Was the Action Better Than Expected?**

$$\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t).$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha_w \delta_t \nabla_{\mathbf{w}} V_w(S_t).$$

**Actor: Modify the Control Policy**

$$\theta \leftarrow \theta + \alpha_\theta \delta_t \nabla_\theta \log \pi_\theta(A_t|S_t).$$

- ▶  $\delta_t > 0$ : make the selected action more likely when the vehicle is in a similar state.
- ▶  $\delta_t < 0$ : make the selected action less likely in similar states.



# Cruise Control Using Actor–Critic



**Critic: Was the Action Better Than Expected?**

$$\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t).$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha_w \delta_t \nabla_{\mathbf{w}} V_w(S_t).$$

**Actor: Modify the Control Policy**

$$\theta \leftarrow \theta + \alpha_\theta \delta_t \nabla_\theta \log \pi_\theta(A_t|S_t).$$

- ▶  $\delta_t > 0$ : make the selected action more likely when the vehicle is in a similar state.
- ▶  $\delta_t < 0$ : make the selected action less likely in similar states.

## Learning Outcome

The actor learns the control action; the critic learns whether that action improves speed tracking without unnecessarily large inputs.



## **Parallel Actor-Critic Methods: A2C and A3C**

---

# Why Move Beyond a Single Actor–Critic Worker?

## Single environment

- ▶ Generates one trajectory at a time.
- ▶ Consecutive transitions can be highly correlated.
- ▶ Explores only one region of the environment at a time.



# Why Move Beyond a Single Actor–Critic Worker?

## Single environment

- ▶ Generates one trajectory at a time.
- ▶ Consecutive transitions can be highly correlated.
- ▶ Explores only one region of the environment at a time.

## Parallel environments

- ▶ Generate several independent trajectories.
- ▶ Provide more diverse training experience.
- ▶ Improve data-collection speed.



# Why Move Beyond a Single Actor–Critic Worker?

## Single environment

- ▶ Generates one trajectory at a time.
- ▶ Consecutive transitions can be highly correlated.
- ▶ Explores only one region of the environment at a time.

## Parallel environments

- ▶ Generate several independent trajectories.
- ▶ Provide more diverse training experience.
- ▶ Improve data-collection speed.



# Why Move Beyond a Single Actor–Critic Worker?

## Single environment

- ▶ Generates one trajectory at a time.
- ▶ Consecutive transitions can be highly correlated.
- ▶ Explores only one region of the environment at a time.

## Parallel environments

- ▶ Generate several independent trajectories.
- ▶ Provide more diverse training experience.
- ▶ Improve data-collection speed.



## Core Idea

A2C and A3C use the same actor–critic principle, but learn from multiple environment instances running in parallel.



# Common Learning Process in A2C and A3C

## 1. Create parallel workers

- Each worker interacts with an independent copy of the environment.



# Common Learning Process in A2C and A3C

1. **Create parallel workers**
  - Each worker interacts with an independent copy of the environment.
2. **Use the current actor and critic**
  - Workers select actions using the current policy and evaluate states using the critic.



# Common Learning Process in A2C and A3C

## 1. Create parallel workers

- Each worker interacts with an independent copy of the environment.

## 2. Use the current actor and critic

- Workers select actions using the current policy and evaluate states using the critic.

## 3. Collect n-step rollouts

- Each worker collects states, actions and rewards for a fixed number of steps.



# Common Learning Process in A2C and A3C

1. **Create parallel workers**
  - Each worker interacts with an independent copy of the environment.
2. **Use the current actor and critic**
  - Workers select actions using the current policy and evaluate states using the critic.
3. **Collect n-step rollouts**
  - Each worker collects states, actions and rewards for a fixed number of steps.
4. **Evaluate the selected actions**
  - The rollout and critic are used to estimate the advantage of each action.



# Common Learning Process in A2C and A3C

1. **Create parallel workers**
  - Each worker interacts with an independent copy of the environment.
2. **Use the current actor and critic**
  - Workers select actions using the current policy and evaluate states using the critic.
3. **Collect n-step rollouts**
  - Each worker collects states, actions and rewards for a fixed number of steps.
4. **Evaluate the selected actions**
  - The rollout and critic are used to estimate the advantage of each action.
5. **Improve both networks**
  - The actor improves action selection; the critic improves value prediction.

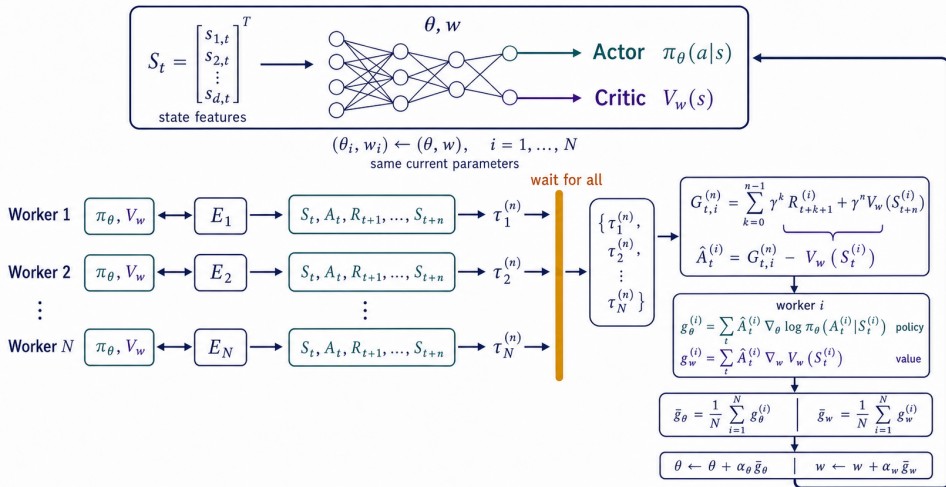


# Common Learning Process in A2C and A3C

1. **Create parallel workers**
  - Each worker interacts with an independent copy of the environment.
2. **Use the current actor and critic**
  - Workers select actions using the current policy and evaluate states using the critic.
3. **Collect n-step rollouts**
  - Each worker collects states, actions and rewards for a fixed number of steps.
4. **Evaluate the selected actions**
  - The rollout and critic are used to estimate the advantage of each action.
5. **Improve both networks**
  - The actor improves action selection; the critic improves value prediction.
6. **Repeat with updated parameters**
  - Workers continue generating experience using newer versions of the shared model.



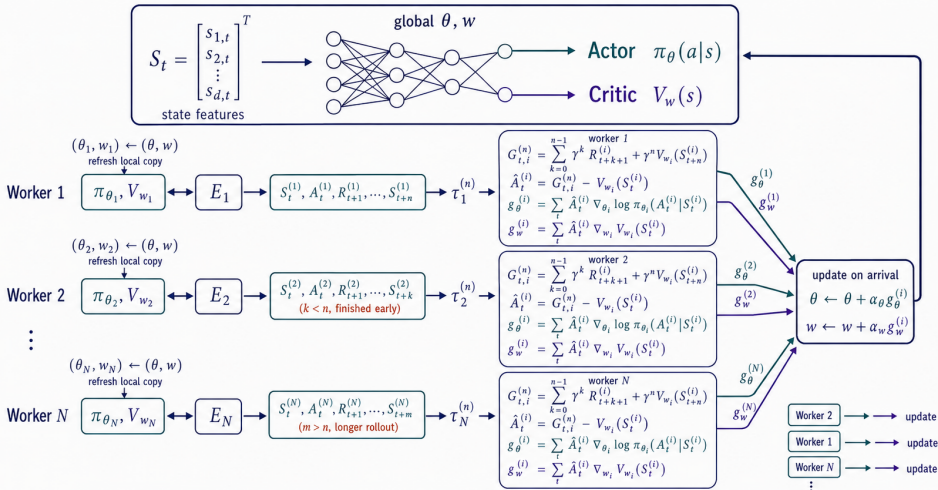
# A2C: Synchronous Advantage Actor–Critic



**SYNCHRONOUS:** every worker finishes before the shared actor and critic are updated.



# A3C: Asynchronous Advantage Actor Critic



**ASYNCHRONOUS:** each worker updates the shared actor and critic as soon as its rollout finishes.



# A2C and A3C: What Is the Difference?

| Aspect         | A2C                                             | A3C                                            |
|----------------|-------------------------------------------------|------------------------------------------------|
| Coordination   | Synchronous                                     | Asynchronous                                   |
| Update trigger | All workers finish their rollouts               | Any worker finishes its rollout                |
| Gradients      | Averaged across workers before updating         | Applied separately by each worker              |
| Waiting        | Workers wait for the slowest worker             | Workers continue independently                 |
| Parameters     | All workers use the same version during a batch | Workers may temporarily use different versions |
| Implementation | Convenient for batched computation              | Requires asynchronous shared updates           |



# When Should We Prefer A2C or A3C?

Both methods can address the same control problems. The choice depends mainly on the available computing resources and environment execution.

## Prefer A2C When

- ▶ Workers complete rollouts at similar speeds.

## Prefer A3C When

- ▶ Worker completion times vary considerably.



# When Should We Prefer A2C or A3C?

Both methods can address the same control problems. The choice depends mainly on the available computing resources and environment execution.

## Prefer A2C When

- ▶ Workers complete rollouts at similar speeds.
- ▶ Experience can be processed efficiently in batches.

## Prefer A3C When

- ▶ Worker completion times vary considerably.
- ▶ Waiting for slower workers would waste time.



# When Should We Prefer A2C or A3C?

Both methods can address the same control problems. The choice depends mainly on the available computing resources and environment execution.

## Prefer A2C When

- ▶ Workers complete rollouts at similar speeds.
- ▶ Experience can be processed efficiently in batches.
- ▶ GPU acceleration is available.

## Prefer A3C When

- ▶ Worker completion times vary considerably.
- ▶ Waiting for slower workers would waste time.
- ▶ Multiple CPU cores are available.



# When Should We Prefer A2C or A3C?

Both methods can address the same control problems. The choice depends mainly on the available computing resources and environment execution.

## Prefer A2C When

- ▶ Workers complete rollouts at similar speeds.
- ▶ Experience can be processed efficiently in batches.
- ▶ GPU acceleration is available.
- ▶ Predictable updates and simpler implementation are preferred.

## Prefer A3C When

- ▶ Worker completion times vary considerably.
- ▶ Waiting for slower workers would waste time.
- ▶ Multiple CPU cores are available.
- ▶ Some parameter and gradient staleness is acceptable.



# When Should We Prefer A2C or A3C?

Both methods can address the same control problems. The choice depends mainly on the available computing resources and environment execution.

## Prefer A2C When

- ▶ Workers complete rollouts at similar speeds.
- ▶ Experience can be processed efficiently in batches.
- ▶ GPU acceleration is available.
- ▶ Predictable updates and simpler implementation are preferred.
- ▶ **Potential Use Case:** Cruise-control policy trained using many identical vehicle simulators on a GPU.

## Prefer A3C When

- ▶ Worker completion times vary considerably.
- ▶ Waiting for slower workers would waste time.
- ▶ Multiple CPU cores are available.
- ▶ Some parameter and gradient staleness is acceptable.
- ▶ **Potential Use Case:** Warehouse-robot navigation trained across CPU workers running environments of different complexity.



## Summary So far...

- ▶ Actor-critic methods learn a **policy** and a **value function** together.
- ▶ The **actor** selects actions, while the **critic** evaluates the outcomes of those actions.
- ▶ The TD error provides an estimate of **advantage**: how much better or worse an action performed than expected in that state.
- ▶ Positive advantage makes the selected action more likely in similar states; negative advantage makes it less likely.
- ▶ Bootstrapping enables updates before an episode ends and generally reduces variance, but introduces bias through the critic's approximation.
- ▶ A2C and A3C accelerate training using parallel workers: A2C combines updates synchronously, whereas A3C applies them asynchronously.



# Motivation for Proximal Policy Optimization

A2C and A3C improve how experience is collected and how actor–critic updates are coordinated. However, they do not directly control the size of the resulting policy change.

$$\theta_{\text{new}} = \theta_{\text{old}} + \alpha \hat{A}_t \nabla_{\theta} \log \pi_{\theta_{\text{old}}}(A_t | S_t)$$

- $\hat{A}_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$  indicates the direction in which the policy parameters should change.



# Motivation for Proximal Policy Optimization

A2C and A3C improve how experience is collected and how actor–critic updates are coordinated. However, they do not directly control the size of the resulting policy change.

$$\theta_{\text{new}} = \theta_{\text{old}} + \alpha \hat{A}_t \nabla_{\theta} \log \pi_{\theta_{\text{old}}}(A_t | S_t)$$

- ▶  $\hat{A}_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$  indicates the direction in which the policy parameters should change.
- ▶ **Issue:** Because the policy is represented by a nonlinear neural network, a small parameter update can sometimes produce a large behavioural change:

$$\|\theta_{\text{new}} - \theta_{\text{old}}\| \text{ small} \quad \nRightarrow \quad |\pi_{\theta_{\text{new}}}(a | s) - \pi_{\theta_{\text{old}}}(a | s)| \text{ small.}$$

- ▶ An excessively large change in action probabilities can destroy useful behaviour learnt over time and cause a sudden drop in return.



# Motivation for Proximal Policy Optimization

A2C and A3C improve how experience is collected and how actor–critic updates are coordinated. However, they do not directly control the size of the resulting policy change.

$$\theta_{\text{new}} = \theta_{\text{old}} + \alpha \hat{A}_t \nabla_{\theta} \log \pi_{\theta_{\text{old}}}(A_t | S_t)$$

- ▶  $\hat{A}_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)$  indicates the direction in which the policy parameters should change.
- ▶ **Issue:** Because the policy is represented by a nonlinear neural network, a small parameter update can sometimes produce a large behavioural change:

$$\|\theta_{\text{new}} - \theta_{\text{old}}\| \text{ small} \quad \nRightarrow \quad |\pi_{\theta_{\text{new}}}(a | s) - \pi_{\theta_{\text{old}}}(a | s)| \text{ small.}$$

- ▶ An excessively large change in action probabilities can destroy useful behaviour learnt over time and cause a sudden drop in return.

## Motivation for PPO

How can we improve the actor while explicitly tracking and controlling how far its behaviour moves from the policy before the update?



## Example: When a Good Update Changes the Policy Too Much

- Consider a vehicle travelling below its desired speed on a straight road. Suppose during the rollout, an exploratory action of stronger throttle reduces the speed-tracking error more quickly and produces a higher return than expected:

$$\hat{A}_t(s, \text{strong throttle}) > 0.$$

- The actor update therefore increases its probability:  $\theta_{\text{new}} = \theta_{\text{old}} + \alpha \hat{A}_t \nabla_{\theta} \log \pi_{\theta_{\text{old}}}(A_t | S_t)$ .



## Example: When a Good Update Changes the Policy Too Much

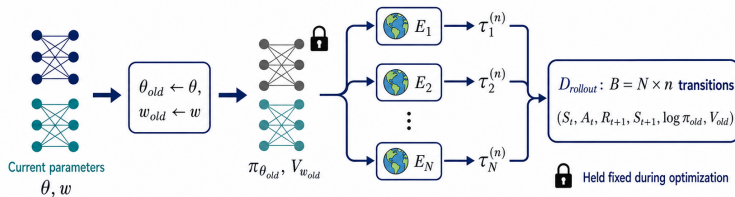
- Consider a vehicle travelling below its desired speed on a straight road. Suppose during the rollout, an exploratory action of stronger throttle reduces the speed-tracking error more quickly and produces a higher return than expected:

$$\hat{A}_t(s, \text{strong throttle}) > 0.$$

- The actor update therefore increases its probability:  $\theta_{\text{new}} = \theta_{\text{old}} + \alpha \hat{A}_t \nabla_{\theta} \log \pi_{\theta_{\text{old}}}(A_t | S_t)$ .
- The rollout based on old policy contains mostly low- and medium-speed states but after an aggressive update, the agent frequently reaches high-speed states that were rarely observed in the rollout.
- **Issue:** The collected experience contains little evidence about braking at high speed, approaching curves or possible instability.
- **Key Insight:** The gradient correctly says “increase strong throttle,” but it does not say how far its probability should increase. A locally beneficial direction can therefore produce an unsafe behavioural change.



# PPO: Collecting the Rollout Batch



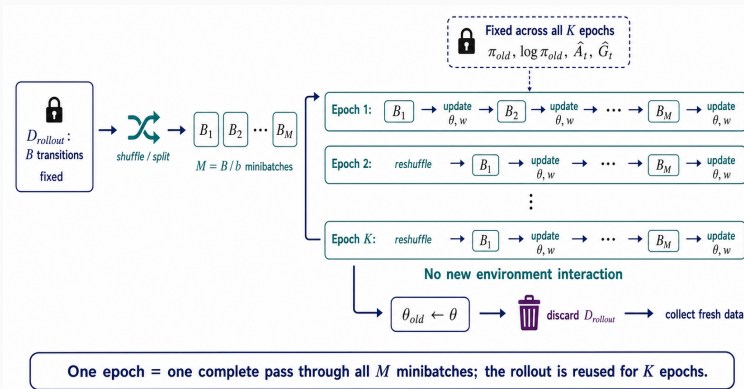
Collect once:  $N$  environments contribute samples to one rollout batch.

## Key Idea

A snapshot of the current policy is first frozen as  $\pi_{\theta_{old}}$ . Using this frozen policy,  $N$  workers are rolled out in parallel, with each worker collecting  $n$  transitions. These samples are combined into one rollout batch containing  $B = Nn$  transitions.



# PPO: Reusing the Rollout for $K$ Epochs



## Key Idea

Each epoch is one complete pass through all rollout minibatches. The current policy is updated after every minibatch, but fresh experience is collected only after completing all  $K$  epochs.



# From Rollout Reuse to Policy Comparison

During each minibatch update, PPO compares the current policy with this frozen policy by computing this ratio:

$$r_t(\theta) = \frac{\pi_{\theta}(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}.$$

- ▶  $\pi_{\theta_{\text{old}}}(A_t | S_t)$  (Action taken) was recorded during rollout collection and remains fixed.
- ▶  $\pi_{\theta}(A_t | S_t)$  (Action based on current policy) is recomputed for every transition in the minibatch.



# From Rollout Reuse to Policy Comparison

During each minibatch update, PPO compares the current policy with this frozen policy by computing this ratio:

$$r_t(\theta) = \frac{\pi_{\theta}(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}.$$

- ▶  $\pi_{\theta_{\text{old}}}(A_t | S_t)$  (Action taken) was recorded during rollout collection and remains fixed.
- ▶  $\pi_{\theta}(A_t | S_t)$  (Action based on current policy) is recomputed for every transition in the minibatch.
- ▶ After each minibatch update,  $\theta$  changes; hence the ratio is recomputed after every minibatch update.



# From Rollout Reuse to Policy Comparison

During each minibatch update, PPO compares the current policy with this frozen policy by computing this ratio:

$$r_t(\theta) = \frac{\pi_{\theta}(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}.$$

- ▶  $\pi_{\theta_{\text{old}}}(A_t | S_t)$  (Action taken) was recorded during rollout collection and remains fixed.
- ▶  $\pi_{\theta}(A_t | S_t)$  (Action based on current policy) is recomputed for every transition in the minibatch.
- ▶ After each minibatch update,  $\theta$  changes; hence the ratio is recomputed after every minibatch update.

$r_t(\theta) = 1$  : sampled action has the same probability,  
 $r_t(\theta) > 1$  : sampled action has become more probable,  
 $r_t(\theta) < 1$  : sampled action has become less probable.



# From Rollout Reuse to Policy Comparison

During each minibatch update, PPO compares the current policy with this frozen policy by computing this ratio:

$$r_t(\theta) = \frac{\pi_{\theta}(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}.$$

- ▶  $\pi_{\theta_{\text{old}}}(A_t | S_t)$  (Action taken) was recorded during rollout collection and remains fixed.
- ▶  $\pi_{\theta}(A_t | S_t)$  (Action based on current policy) is recomputed for every transition in the minibatch.
- ▶ After each minibatch update,  $\theta$  changes; hence the ratio is recomputed after every minibatch update.

$$\begin{aligned} r_t(\theta) = 1 & : \text{ sampled action has the same probability,} \\ r_t(\theta) > 1 & : \text{ sampled action has become more probable,} \\ r_t(\theta) < 1 & : \text{ sampled action has become less probable.} \end{aligned}$$

## Intuition of $r_t(\theta)$

The intuition is that  $r_t(\theta)$  measures how much the current policy has shifted its opinion about a specific action.

## What Is Optimized?

The ratio is not optimized by itself. It enters the PPO optimization objective, which determines how  $\theta$  is updated.



# PPO's Unclipped Objective

- ▶ With the ratio in hand, we can write an objective that scores the new policy using old data and maximise it for each minibatch. For a minibatch  $\mathcal{B}_m$  containing  $b$  transitions, the objective is:

$$L_m(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} r_j(\theta) \hat{A}_j$$

- ▶ Advantage  $\hat{A}_t$  says whether the action was better than average or worse relative to the old policy.
- ▶ The ratio  $r_t(\theta)$  says how the new policy is treating that action relative to the old one.



# PPO's Unclipped Objective

- ▶ With the ratio in hand, we can write an objective that scores the new policy using old data and maximise it for each minibatch. For a minibatch  $\mathcal{B}_m$  containing  $b$  transitions, the objective is:

$$L_m(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} r_j(\theta) \hat{A}_j$$

- ▶ Advantage  $\hat{A}_t$  says whether the action was better than average or worse relative to the old policy.
- ▶ The ratio  $r_t(\theta)$  says how the new policy is treating that action relative to the old one.
- ▶ **Intuition of multiplying  $\hat{A}_t$  and  $r_t(\theta)$ :** if an action was good, increasing its ratio raises the objective; if an action was bad, increasing its ratio lowers the objective



# PPO's Unclipped Objective

- ▶ With the ratio in hand, we can write an objective that scores the new policy using old data and maximise it for each minibatch. For a minibatch  $\mathcal{B}_m$  containing  $b$  transitions, the objective is:

$$L_m(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} r_j(\theta) \hat{A}_j$$

- ▶ Advantage  $\hat{A}_t$  says whether the action was better than average or worse relative to the old policy.
- ▶ The ratio  $r_t(\theta)$  says how the new policy is treating that action relative to the old one.
- ▶ **Intuition of multiplying  $\hat{A}_t$  and  $r_t(\theta)$ :** if an action was good, increasing its ratio raises the objective; if an action was bad, increasing its ratio lowers the objective
- ▶ **Connection to PPO motivation:** This objective explicitly evaluates each policy update relative to the frozen policy that generated the rollout. It therefore makes both the direction and magnitude of changes in sampled-action probabilities visible during optimisation.



# PPO's Unclipped Objective

- ▶ With the ratio in hand, we can write an objective that scores the new policy using old data and maximise it for each minibatch. For a minibatch  $\mathcal{B}_m$  containing  $b$  transitions, the objective is:

$$L_m(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} r_j(\theta) \hat{A}_j$$

- ▶ Advantage  $\hat{A}_t$  says whether the action was better than average or worse relative to the old policy.
- ▶ The ratio  $r_t(\theta)$  says how the new policy is treating that action relative to the old one.
- ▶ **Intuition of multiplying  $\hat{A}_t$  and  $r_t(\theta)$ :** if an action was good, increasing its ratio raises the objective; if an action was bad, increasing its ratio lowers the objective
- ▶ **Connection to PPO motivation:** This objective explicitly evaluates each policy update relative to the frozen policy that generated the rollout. It therefore makes both the direction and magnitude of changes in sampled-action probabilities visible during optimisation.
- ▶ The actor parameters  $\theta$  and critic parameters  $\mathbf{w}$  are updated after processing each minibatch.
- ▶ If the rollout is reused for  $K$  epochs, one complete PPO iteration produces  $MK$  updates before fresh experience is collected.



# Why Is the Unclipped Objective Still Unsafe?

For each minibatch, the current policy maximises

$$L_m(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} r_j(\theta) \hat{A}_j.$$

Consider one transition from the rollout and following two scenarios:



# Why Is the Unclipped Objective Still Unsafe?

For each minibatch, the current policy maximises

$$L_m(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} r_j(\theta) \hat{A}_j.$$

Consider one transition from the rollout and following two scenarios:

$\hat{A}_j > 0$ : **Good Action**

The objective increases whenever  $r_j(\theta)$

increases:  $r_j \hat{A}_j \uparrow$  as  $r_j \uparrow$ .



# Why Is the Unclipped Objective Still Unsafe?

For each minibatch, the current policy maximises

$$L_m(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} r_j(\theta) \hat{A}_j.$$

Consider one transition from the rollout and following two scenarios:

$\hat{A}_j > 0$ : **Good Action**

The objective increases whenever  $r_j(\theta)$

increases:  $r_j \hat{A}_j \uparrow$  as  $r_j \uparrow$ .

$\hat{A}_j < 0$ : **Poor Action**

The objective increases as  $r_j(\theta)$  decreases:

$r_j \hat{A}_j \uparrow$  as  $r_j \downarrow$ .



# Why Is the Unclipped Objective Still Unsafe?

For each minibatch, the current policy maximises

$$L_m(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} r_j(\theta) \hat{A}_j.$$

Consider one transition from the rollout and following two scenarios:

$\hat{A}_j > 0$ : **Good Action**

The objective increases whenever  $r_j(\theta)$

increases:

$$r_j \hat{A}_j \uparrow \quad \text{as} \quad r_j \uparrow.$$

$\hat{A}_j < 0$ : **Poor Action**

The objective increases as  $r_j(\theta)$  decreases:

$$r_j \hat{A}_j \uparrow \quad \text{as} \quad r_j \downarrow.$$

## Example

If a good action had old probability 0.15 and following are potential updates:

$$0.15 \rightarrow 0.30 \Rightarrow r_j = 2, \quad 0.15 \rightarrow 0.60 \Rightarrow r_j = 4.$$



# Why Is the Unclipped Objective Still Unsafe?

For each minibatch, the current policy maximises

$$L_m(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} r_j(\theta) \hat{A}_j.$$

Consider one transition from the rollout and following two scenarios:

$\hat{A}_j > 0$ : **Good Action**

The objective increases whenever  $r_j(\theta)$

increases:  $r_j \hat{A}_j \uparrow$  as  $r_j \uparrow$ .

$\hat{A}_j < 0$ : **Poor Action**

The objective increases as  $r_j(\theta)$  decreases:

$r_j \hat{A}_j \uparrow$  as  $r_j \downarrow$ .

## Example

If a good action had old probability 0.15 and following are potential updates:

$$0.15 \rightarrow 0.30 \Rightarrow r_j = 2, \quad 0.15 \rightarrow 0.60 \Rightarrow r_j = 4.$$

**Issue:** The unclipped objective prefers the second change, even though it may alter the policy far more aggressively. Repeated minibatch updates can therefore change the policy excessively.



# PPO's Clipped Objective

PPO first restricts the ratio used by the objective, where typically  $\epsilon_{\text{clip}} \in [0.1, 0.3]$ :

$$\bar{r}_j(\theta) = \text{clip}(r_j(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}).$$



# PPO's Clipped Objective

PPO first restricts the ratio used by the objective, where typically  $\epsilon_{\text{clip}} \in [0.1, 0.3]$ :

$$\bar{r}_j(\theta) = \text{clip}(r_j(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}).$$

It then maximises the clipped minibatch objective

$$L_m^{\text{CLIP}}(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} \min \left[ r_j(\theta) \hat{A}_j, \bar{r}_j(\theta) \hat{A}_j \right]$$



# PPO's Clipped Objective

PPO first restricts the ratio used by the objective, where typically  $\epsilon_{\text{clip}} \in [0.1, 0.3]$ :

$$\bar{r}_j(\theta) = \text{clip}(r_j(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}).$$

It then maximises the clipped minibatch objective

$$L_m^{\text{CLIP}}(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} \min \left[ r_j(\theta) \hat{A}_j, \bar{r}_j(\theta) \hat{A}_j \right]$$

$\hat{A}_j > 0$ : **Good Action**

Increasing the action probability is rewarded

only until

$$r_j(\theta) = 1 + \epsilon_{\text{clip}}.$$

Increasing it further provides no additional objective gain.



# PPO's Clipped Objective

PPO first restricts the ratio used by the objective, where typically  $\epsilon_{\text{clip}} \in [0.1, 0.3]$ :

$$\bar{r}_j(\theta) = \text{clip}(r_j(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}).$$

It then maximises the clipped minibatch objective

$$L_m^{\text{CLIP}}(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} \min \left[ r_j(\theta) \hat{A}_j, \bar{r}_j(\theta) \hat{A}_j \right]$$

$\hat{A}_j > 0$ : **Good Action**

Increasing the action probability is rewarded only until

$$r_j(\theta) = 1 + \epsilon_{\text{clip}}.$$

Increasing it further provides no additional objective gain.

$\hat{A}_j < 0$ : **Poor Action**

Decreasing the action probability is rewarded only until

$$r_j(\theta) = 1 - \epsilon_{\text{clip}}.$$

Decreasing it further provides no additional objective gain.



# PPO's Clipped Objective

PPO first restricts the ratio used by the objective, where typically  $\epsilon_{\text{clip}} \in [0.1, 0.3]$ :

$$\bar{r}_j(\theta) = \text{clip}(r_j(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}).$$

It then maximises the clipped minibatch objective

$$L_m^{\text{CLIP}}(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} \min \left[ r_j(\theta) \hat{A}_j, \bar{r}_j(\theta) \hat{A}_j \right]$$

$\hat{A}_j > 0$ : **Good Action**

Increasing the action probability is rewarded only until

$$r_j(\theta) = 1 + \epsilon_{\text{clip}}.$$

Increasing it further provides no additional objective gain.

**Note:**

Clipping does not force every ratio to remain inside this interval. It removes the incentive to move farther in an already excessive improvement direction.

$\hat{A}_j < 0$ : **Poor Action**

Decreasing the action probability is rewarded only until

$$r_j(\theta) = 1 - \epsilon_{\text{clip}}.$$

Decreasing it further provides no additional objective gain.



# PPO Can Also Use a KL Penalty

An alternative PPO objective directly penalises the difference between the old and current action distributions:

$$L_m^{\text{KL}}(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} \left[ r_j(\theta) \hat{A}_j - \beta D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot | S_j) \parallel \pi_{\theta}(\cdot | S_j)) \right]$$

- ▶ The first term encourages actions with positive advantage and discourages actions with negative advantage.
- ▶  $D_{\text{KL}}$  measures the difference between the old and current action distributions at state  $S_j$ .



# PPO Can Also Use a KL Penalty

An alternative PPO objective directly penalises the difference between the old and current action distributions:

$$\mathcal{L}_m^{\text{KL}}(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} \left[ r_j(\theta) \hat{A}_j - \beta D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot | S_j) \parallel \pi_{\theta}(\cdot | S_j)) \right]$$

- ▶ The first term encourages actions with positive advantage and discourages actions with negative advantage.
- ▶  $D_{\text{KL}}$  measures the difference between the old and current action distributions at state  $S_j$ .
- ▶ Small  $\beta$ : greater freedom to change the policy.
- ▶ Large  $\beta$ : stronger pressure to remain near the old policy.
- ▶  $\beta$  may be adapted to maintain a desired KL divergence.



# PPO Can Also Use a KL Penalty

An alternative PPO objective directly penalises the difference between the old and current action distributions:

$$L_m^{\text{KL}}(\theta) = \frac{1}{b} \sum_{j \in \mathcal{B}_m} \left[ r_j(\theta) \hat{A}_j - \beta D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot | S_j) \parallel \pi_{\theta}(\cdot | S_j)) \right]$$

- ▶ The first term encourages actions with positive advantage and discourages actions with negative advantage.
- ▶  $D_{\text{KL}}$  measures the difference between the old and current action distributions at state  $S_j$ .
- ▶ Small  $\beta$ : greater freedom to change the policy.
- ▶ Large  $\beta$ : stronger pressure to remain near the old policy.
- ▶  $\beta$  may be adapted to maintain a desired KL divergence.

## Two Ways to Stay Proximal

Clipped PPO limits the benefit of excessive probability-ratio changes; KL-penalty PPO directly penalises overall policy-distribution change.



# The Complete PPO Learning Loop

## 1. Freeze the rollout policy:

$$\theta_{\text{old}} \leftarrow \theta.$$

Keep  $\pi_{\theta_{\text{old}}}$  fixed until the current PPO iteration is complete.



# The Complete PPO Learning Loop

1. **Freeze the rollout policy:**

$$\theta_{\text{old}} \leftarrow \theta.$$

Keep  $\pi_{\theta_{\text{old}}}$  fixed until the current PPO iteration is complete.

2. **Collect one rollout batch:** using  $\pi_{\theta_{\text{old}}}$ ,  $N$  workers collect  $n$  transitions each, producing  $B = Nn$  samples. Record  $S_j, A_j, R_{j+1}, \log \pi_{\theta_{\text{old}}}(A_j | S_j), V_{\mathbf{w}_{\text{old}}}(S_j).$



# The Complete PPO Learning Loop

1. **Freeze the rollout policy:**

$$\theta_{\text{old}} \leftarrow \theta.$$

Keep  $\pi_{\theta_{\text{old}}}$  fixed until the current PPO iteration is complete.

2. **Collect one rollout batch:** using  $\pi_{\theta_{\text{old}}}$ ,  $N$  workers collect  $n$  transitions each, producing  $B = Nn$  samples. Record

$$S_j, A_j, R_{j+1}, \log \pi_{\theta_{\text{old}}}(A_j | S_j), V_{\mathbf{w}_{\text{old}}}(S_j).$$

3. **Prepare fixed learning targets:** compute  $\hat{A}_j$  and  $\hat{G}_j$  once for all  $B$  transitions.



# The Complete PPO Learning Loop

1. **Freeze the rollout policy:**

$$\theta_{\text{old}} \leftarrow \theta.$$

Keep  $\pi_{\theta_{\text{old}}}$  fixed until the current PPO iteration is complete.

2. **Collect one rollout batch:** using  $\pi_{\theta_{\text{old}}}$ ,  $N$  workers collect  $n$  transitions each, producing  $B = Nn$  samples. Record

$$S_j, A_j, R_{j+1}, \log \pi_{\theta_{\text{old}}}(A_j | S_j), V_{\mathbf{w}_{\text{old}}}(S_j).$$

3. **Prepare fixed learning targets:** compute  $\hat{A}_j$  and  $\hat{G}_j$  once for all  $B$  transitions.

4. **Optimise for  $K$  epochs:**

- reshuffle the rollout and divide it into  $M$  minibatches;
- for each minibatch, recompute  $\pi_{\theta}(A_j | S_j)$ ,  $r_j(\theta)$  and  $V_{\mathbf{w}}(S_j)$ ;
- update  $\theta$  using the clipped PPO objective (or the KL-penalty alternative);
- update  $\mathbf{w}$  using the critic prediction error.



# The Complete PPO Learning Loop

1. **Freeze the rollout policy:**

$$\theta_{\text{old}} \leftarrow \theta.$$

Keep  $\pi_{\theta_{\text{old}}}$  fixed until the current PPO iteration is complete.

2. **Collect one rollout batch:** using  $\pi_{\theta_{\text{old}}}$ ,  $N$  workers collect  $n$  transitions each, producing  $B = Nn$  samples. Record

$$S_j, A_j, R_{j+1}, \log \pi_{\theta_{\text{old}}}(A_j | S_j), V_{\mathbf{w}_{\text{old}}}(S_j).$$

3. **Prepare fixed learning targets:** compute  $\hat{A}_j$  and  $\hat{G}_j$  once for all  $B$  transitions.

4. **Optimise for  $K$  epochs:**

- reshuffle the rollout and divide it into  $M$  minibatches;
- for each minibatch, recompute  $\pi_{\theta}(A_j | S_j)$ ,  $r_j(\theta)$  and  $V_{\mathbf{w}}(S_j)$ ;
- update  $\theta$  using the clipped PPO objective (or the KL-penalty alternative);
- update  $\mathbf{w}$  using the critic prediction error.

5. **Refresh the data:** after approximately  $MK$  actor-critic updates, discard the rollout and begin the next PPO iteration.



# PPO: Key Takeaways

- ▶ Actor-critic indicates how the policy should improve, but does not directly control the resulting change in action probabilities.
- ▶ PPO freezes the old policy, collects a rollout and compares every subsequent policy update with the policy that generated the data.
- ▶ The importance ratio measures the relative change in probability of each sampled action.
- ▶ The clipped objective removes the incentive for excessively changing action probabilities; a KL penalty provides an alternative.
- ▶ The rollout is divided into  $M$  minibatches and reused for  $K$  epochs, normally producing  $MK$  actor-critic parameter updates.
- ▶ The old policy, sampled data, advantages and return targets remain fixed during these updates; fresh on-policy data is collected afterward.

